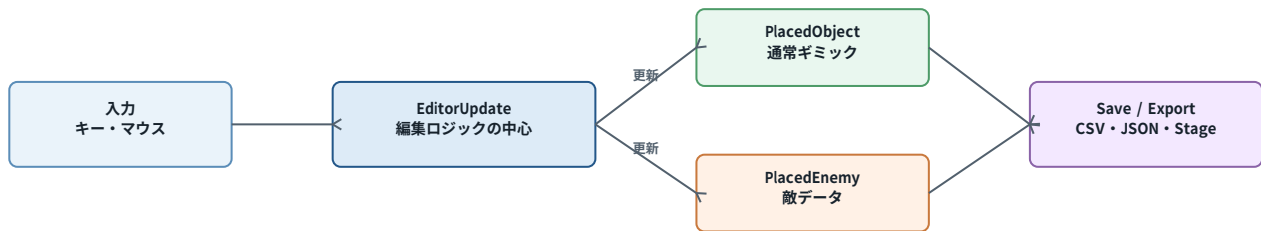


StageEditor.cpp

コード全体の流れ

C++ / raylib 2Dゲーム用ステージエディタ
関数同士のつながりと、1フレーム内の処理順序を図解



このファイルは、入力を受け取って編集用データを変更し、そのデータを保存・実行用ステージ変換へ渡す「編集ロジック本体」です。

作成対象: 提供された StageEditor.cpp 全体

1. 全体構造と責務

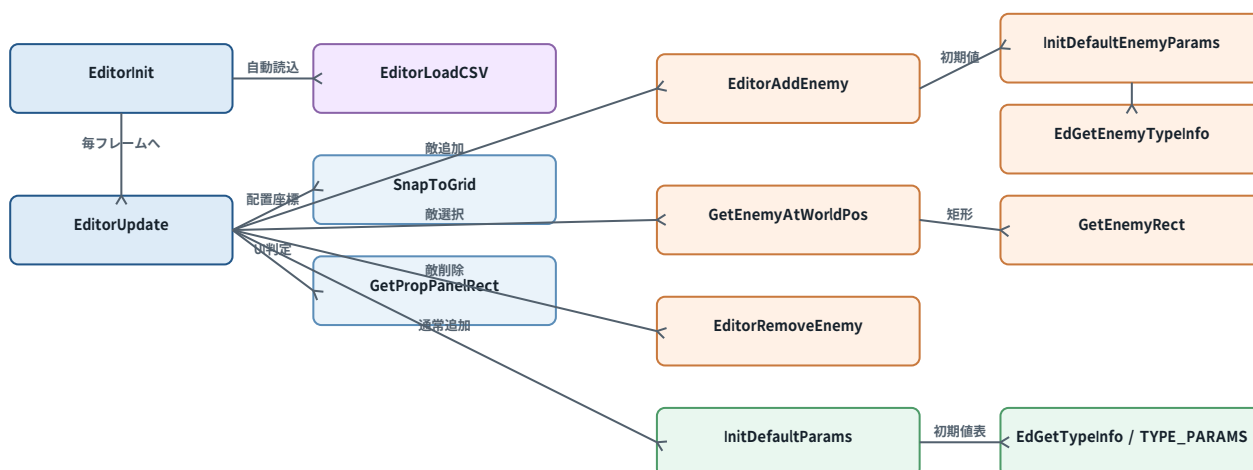
コードは大きく「設定テーブル」「補助関数」「初期化」「1フレーム更新」の4層に分かれます。中心は EditorUpdate で、他の関数はその処理を支える部品です。

層	主な要素	役割
設定テーブル	TYPE_PARAMS / ENEMY_TYPE_PARAMS 名前・色テーブル	種類ごとのパラメータ名、初期値、bool属性、表示名、色を定義
データ操作	InitDefaultParams EditorAddEnemy / EditorRemoveEnemy EditorSetEnemyParam	PlacedObject / PlacedEnemy の追加、削除、初期化、値変更
UI・座標補助	GetPropPanelRect / SnapToGrid GetBtnRect / AdjustToolbarOffset GetEnemyAtWorldPos	当たり判定、UI矩形、グリッド吸着、ツールバー表示範囲を計算
実行入口	EditorInit / EditorUpdate	初期状態の作成と、毎フレームの入力処理

重要な設計ルール

- type はギミックの挙動を識別する。
- spriteId / rotation / flipX / flipY は見た目だけを変更する。
- params[0..5] は type ごとに意味が異なるため、保存・読込・実行変換側と順序を一致させる。
- 通常オブジェクトは ed.objects、敵は ed.placedEnemies に分けて保持する。

2. 関数同士のつながり



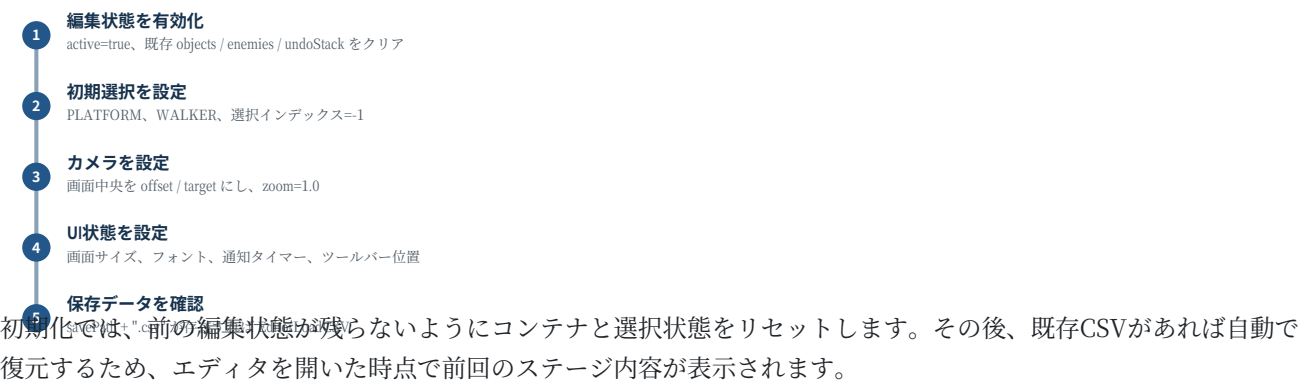
上図の通り、EditorUpdate が入力に応じて各補助関数を呼びます。EditorInit は開始時に状態を初期化し、保存済みCSVが存在すれば EditorLoadCSV を呼びます。

敵関連の代表的な呼び出し連鎖

■■■■■ → EditorAddEnemy → InitDefaultEnemyParams → EdGetEnemyTypeInfo → ENEMY_TYPE_PARAMS

T■■ / ■■■■■ → GetEnemyAtWorldPos → GetEnemyRect → CheckCollisionPointRec

3. EditorInit: 編集開始時の流れ



主な初期値

項目	初期値	意味
currentType	PLATFORM	最初に通常床を選択
selectedEnemyIdx / propSelectedIdx	-1	何も選択されていない
currentEnemyType	WALKER	最初の敵タイプ
camera.zoom	1.0	標準倍率
saveNotifyTimer	0.0	保存通知を非表示

4. EditorUpdate: 1フレーム全体の処理順序

- 1 **早期終了・状態補正**
active確認、通知タイマー減算、無効な選択番号を解除
- 2 **編集モードを最優先**
敵数値 → 通常数値 → コメント文字列。処理後 return
- 3 **プロパティパネル操作**
行クリック、bool切替、数値編集開始、削除、コメント編集開始
- 4 **見た目・複製・サイズ**
spriteId、反転、回転、コピー、貼付、拡大縮小
- 5 **通常入力**
種類選択、グリッド、配置サイズ、カメラ移動、ズーム
- 6 **ステージ領域操作**
ドラッグ、左配置、右削除、T選択

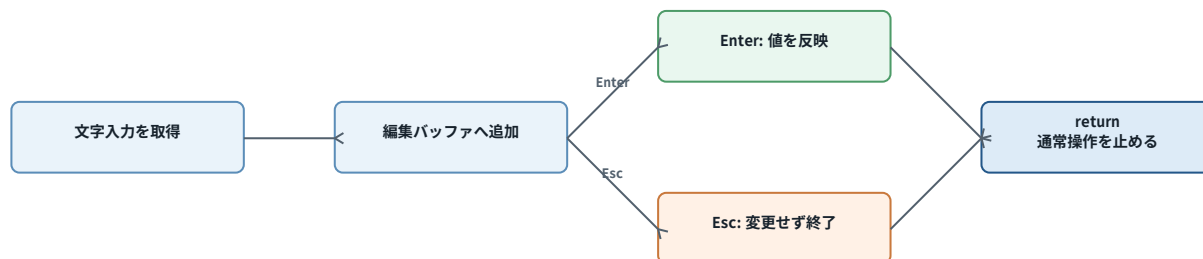
最重要点: 数値入力やコメント入力の処理は通常操作より前に置かれ、最後に return; します。これにより、文字入力中に同じキーが配置・サイズ変更・保存以外の操作として誤反応するのを防ぎます。

処理順を変更する場合の注意

- 編集モードの return を外すと、入力したキーが通常ショートカットにも伝わる。
- UI判定より先に配置処理を行うと、パネルをクリックしただけで背後にオブジェクトが置かれる。
- erase 後に選択インデックスを補正しないと、別オブジェクトを参照したり範囲外アクセスにつながる。

5. 編集モードの3分岐

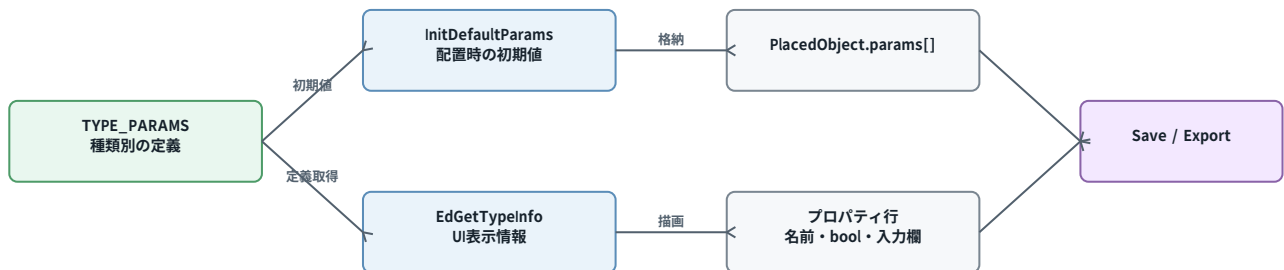
編集モード	開始条件	入力処理	終了条件
敵パラメータ	currentType=ENEMY selectedEnemyIdx>=0 propEditingParam>=0	数字・小数点・マイナス Enterで EditorSetEnemyParam	Enter確定 Esc中止
通常パラメータ	propSelectedIdx>=0 propEditingParam>=0	数字入力を propEditBuf に蓄積 Enterで params[] に反映	Enter / Esc パネル外クリック
コメント文字列	propSelectedIdx>=0 propEditingText=true	UTF-8入力、Backspace Ctrl+V貼付	Enter / Esc パネル外クリック



コメント入力では、日本語1文字が複数バイトになるためUTF-8へ変換して保存します。Backspace時も途中バイトを避け、1文字単位で削除します。

6. プロパティパネルとパラメータテーブル

TYPE_PARAMS と ENEMY_TYPE_PARAMS は、表示名・初期値・bool判定を一元管理します。オブジェクトを配置したときの初期化と、プロパティパネルの表示内容が同じ定義を参照します。

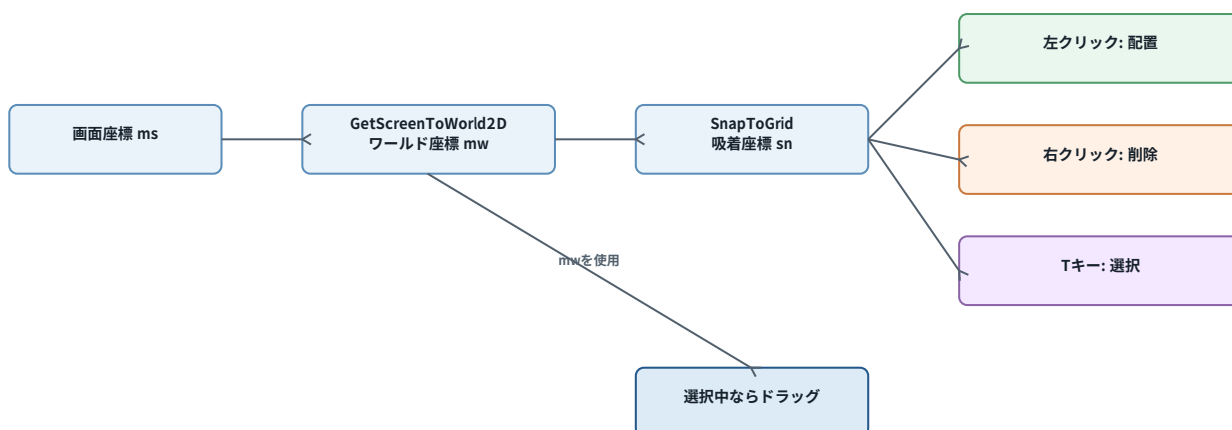


例: ELEVATOR は `params[0]=speed`、`params[1]=rangeUp`、`params[2]=rangeDn` として定義されています。この順番を後から変更すると、既存CSVに保存された値の意味が変わるため注意が必要です。

GetPropPanelRect の役割

- 選択中オブジェクトのパラメータ数からパネル高さを計算する。
- COMMENT_BLOCK の場合はテキスト入力行とプレビュー行を追加する。
- `spriteId / rotation / flip` の表示分として3行分を加算する。
- EditorUpdate 内のクリック判定と描画側のサイズを一致させる。

7. ステージ領域での操作フロー



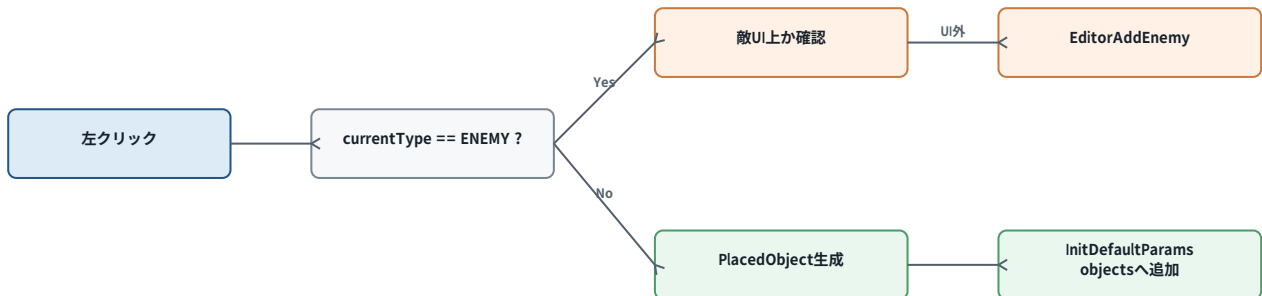
ステージ領域として扱われるのは、ツールバー・下部UI・通常プロパティパネルの外側です。敵配置ではさらに敵サイドパネルと敵プロパティパネルを個別に除外します。

ドラッグ移動

- 選択中オブジェクト上で左クリックすると isDragging=true。
- クリックした位置とオブジェクト左上との差を dragStart に保持する。
- 押下中は新しい左上座標を計算し、SnapToGrid 後に rect.x / rect.y を更新する。
- ボタンを離すと isDragging=false。配置処理は !isDragging のときだけ行う。

8. 左クリック配置: 通常オブジェクトと敵

対象	呼び出し・処理	最終的な格納先
通常オブジェクト	PlacedObject作成 → type / rect設定 → InitDefaultParams	ed.objects.push_back(newObj)
敵	UI上でないことを確認 → EditorAddEnemy → InitDefaultEnemyParams	ed.placedEnemies.push_back(e)



敵追加後は `selectedEnemyIdx = placedEnemies.size() - 1` とし、新しく置いた敵をすぐ選択状態にします。これにより、配置直後に右側のプロパティを編集できます。

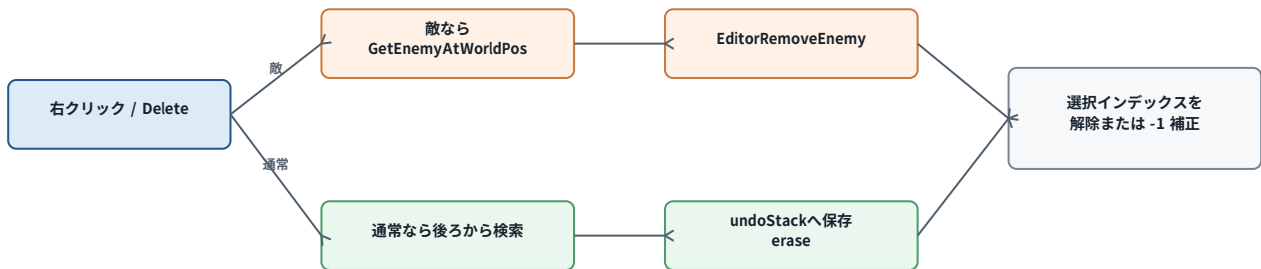
通常オブジェクトの大きさ

```
width = gridSize × gridW
height = gridSize × gridH
```

Q/Eで横マス数、W/Sで縦マス数、Rでグリッド幅を変更するため、同じ配置処理でも任意の矩形サイズを作れます。

9. 右クリック・Deleteによる削除と選択インデックス

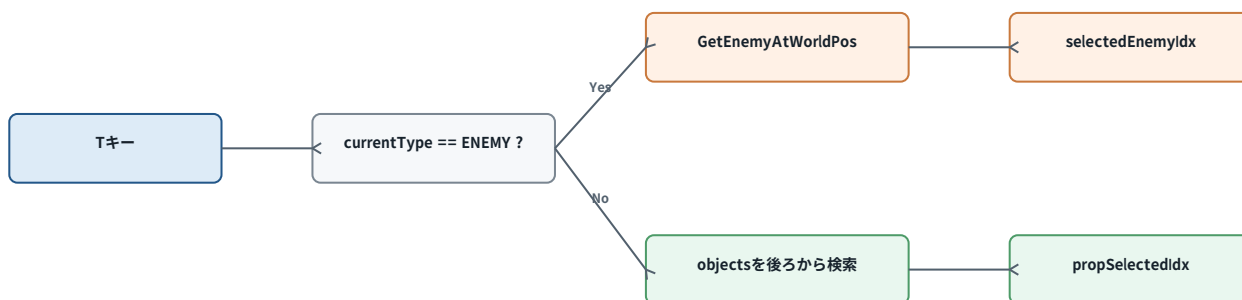
通常オブジェクトは配列の後ろから検索します。後から追加された要素を先に調べるため、重なっている場合に手前側を削除しやすくなります。



削除位置 idx と選択位置	必要な処理	理由
selected == idx	選択を -1 にする	選択対象そのものが消えた
selected > idx	selected--	eraseで後ろの要素が1つ前へ詰まる
selected < idx	変更なし	削除位置より前の番号は変化しない

通常オブジェクトは削除前に undoStack へコピーします。敵削除は EditorRemoveEnemy 内で履歴保存をしていないため、現在の Ctrl+Z では敵を復元できません。

10. Tキーによる選択とプロパティ表示



対象が見つからない場合は選択番号を -1 にして、プロパティ編集状態も解除します。通常オブジェクト側は found フラグで判定し、敵側は GetEnemyAtWorldPos が返す -1 をそのまま利用します。

後ろから検索する理由

objects / placedEnemies の末尾ほど後から追加された要素です。逆順検索して最初に当たった要素を採用し、`break;` することで、重なった要素のうち優先度の高い1つだけを選択します。

状態管理をより明確にするなら、敵を選んだときに `propSelectedIdx=-1`、通常オブジェクトを選んだときに `selectedEnemyIdx=-1` として、同時選択状態を防ぐ方法があります。

11. 見た目変更・コピー・サイズ変更

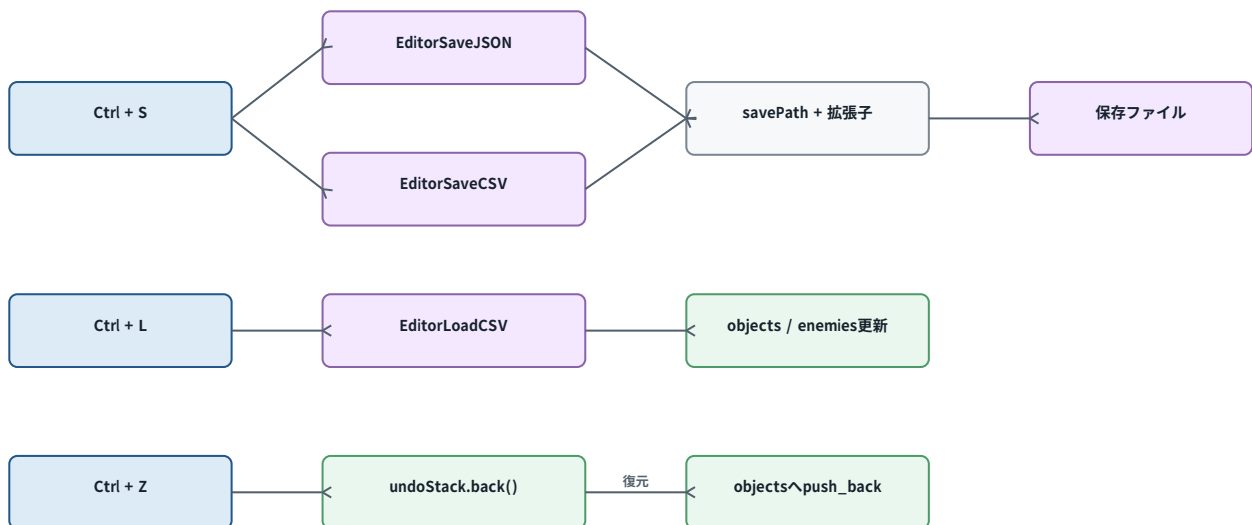
操作	変更対象	挙動への影響
[/]	spriteId	見た目の画像だけを前後に切替
H / J	flipX / flipY	左右・上下反転のみ
K / L	rotation	90度単位で回転、0〜360に正規化
C / P	clipboard / objects	選択物をコピーし、30pxずらして追加
- / =	rect.width / height	10px単位で縮小・拡大。最小10px

これらの操作は、数値やコメントの編集中には前段で return しているため反応しません。また type と params を直接変えないため、ギミックの挙動を保ったまま外観だけを調整できます。

コピー・貼り付けの流れ

```
C: ed.clipboard = sel; ed.hasClipboard = true;
P: PlacedObject newObj = ed.clipboard; x += 30; y += 30; objects.push_back(newObj);
```

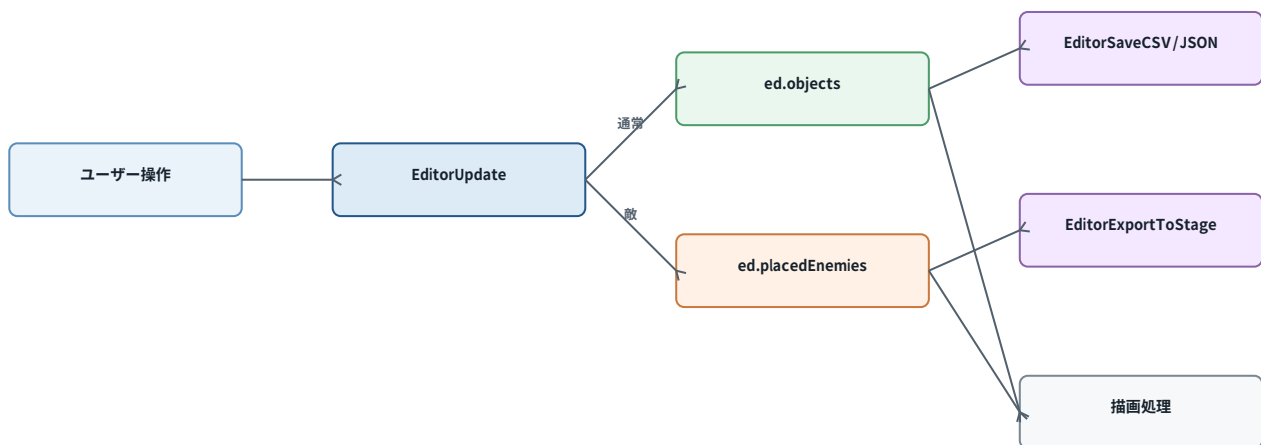
12. 保存・読込・Undoのつながり



- Ctrl+S: JSONとCSVを両方保存し、saveNotifyTimer=2.0で保存通知を出す。
- Ctrl+L: CSVから編集データを読み込む。
- Ctrl+Z: undoStackの末尾を objects の末尾へ戻し、pop_backする。

現在のUndoは「削除した通常オブジェクト」を戻す仕組みです。元の配列番号は保存していないため、画面上の座標は戻っても重なり順が変わる場合があります。また、追加・移動・パラメータ変更・敵削除は履歴対象外です。

13. データが実行・保存へ渡るまで



EditorUpdate が変更するのは、ゲーム本編で直接動くデータではなく、編集用の PlacedObject / PlacedEnemy です。保存時はファイルへ、プレイテスト時は ExportToStage を通じて実行用 Stage へ変換されます。

この分離の利点

- エディタ用UI情報とゲーム実行時の状態を分けられる。
- CSVを再読込して同じステージを編集し直せる。
- 配置データを保ったまま、ゲーム側の実装を変更しやすい。
- spriteIdなどの見た目情報とtypeによる挙動を独立して扱える。

14. 改善するとよい点と保守上の注意

項目	現状	改善案
Undo	通常オブジェクト削除のみ、元インデックスなし	操作種別・対象・元インデックスを持つ UndoAction を導入
敵と通常の選択	片方を選んでも他方の選択番号が残る可能性	選択時に反対側を -1 にして排他的に管理
右Ctrl	KEY_LEFT_CONTROL のみ	左または右Ctrlを ctrlDown にまとめる
読込後の状態	選択・Undo履歴が残る可能性	Load後に選択解除、編集解除、履歴clear
UI当たり判定	通常と敵で判定方法が分散	IsMouseOverEditorUI のような共通関数へ集約
定数	280, 58, 28, 40など直値	PROP_W等の定数へ統一し描画側と共有

最も優先度が高いのは、Undoの共通化 と 選択状態の排他管理

です。これらを整えると、敵・通常オブジェクト・移動・編集値変更を同じ仕組みで扱いやすくなります。

まとめ

この StageEditor.cpp は、設定テーブルを基盤に、EditorInit で編集状態を準備し、EditorUpdate が入力の優先順位を管理しながら objects / placedEnemies を更新する構成です。編集モードの早期 return、UI領域の除外、erase後のインデックス補正が、誤操作と不正参照を防ぐ中心的な仕組みです。